UNIVERSIDAD POLITÉCNICA DE PACHUCA

DESARROLLO E IMPLEMENTACIÓN DE COLUMNAS Y LAYOUTS EN JETPACK COMPOSE

Curso: Aplicaciones Móviles / Desarrollo Móvil

Autor: Cervantes Gonzalez Johan Baru

Profesor: Montoya Cerro Miguel Angel

Lugar y Fecha: Zempoala, Hidalgo, 05/10/2026



Índice

1. Introducción **4**
1.1. Contextualización 4
1.2. Planteamiento del Problema 4
1.3. Objetivos 4
1.3.1. Objetivo General 4
1.3.2. Objetivos Específicos 4
1.4. Justificación 4

2. Marco Teórico **5**
2.1. Definición de Conceptos Clave 5
2.2. Teorías y Antecedentes 5

3. Desarrollo **6**
3.1. Análisis del Cambio de Paradigma: Column vs. LinearLayout 6
3.2. Guía Paso a Paso y Desarrollo de la Práctica 6
3.3. Explicación Narrativa del Comportamiento Visual y Estructura 11

4. Conclusiones **12**

5. Anexos **13**

6. Referencias **14**

Índice de figuras

1. Creación del proyecto “column application” en Android Studio 6
2. Configuración inicial de la función composable y vista previa 7
3. Comprobación del encimado de textos al no usar un contenedor de maquetación 7
4. Organización vertical exitosa de los elementos mediante Column 8
5. Aplicación del modificador para expandir la columna a pantalla completa 8
6. Asignación de color de fondo al contenedor mediante modificadores . . 9
7. Incorporación del archivo de imagen WebP en la carpeta drawable . . . 9
8. Visualización de la imagen integrada en la jerarquía vertical 10
9. Personalización de tamaño y color en los textos de la interfaz 10
10. Centrado del título mediante expansión de ancho y alineación de texto 11

Índice de tablas

1. Comparativa entre Column en Compose y LinearLayout vertical 6

1. Introducción

1.1. Contextualización

En mi experiencia desarrollando aplicaciones Android, el paso al paradigma declarativo con Jetpack Compose exige entender a profundidad cómo estructurar elementos visuales de manera eficiente. En este reporte explico el funcionamiento detallado de la maquetación vertical mediante componentes esenciales como columnas estándar y columnas de carga perezosa.

1.2. Planteamiento del Problema

Observé que en Jetpack Compose, al colocar múltiples componentes visuales sin establecer un contenedor de diseño explícito, estos se superponen unos sobre otros en el origen del lienzo. Para resolver esta problemática y lograr interfaces ordenadas, requerí analizar la organización en columnas, la aplicación secuencial de modificadores de tamaño, color y alineación, así como la gestión avanzada del desplazamiento ante cambios de orientación mediante listas perezosas.

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar una aplicación móvil enfocada en el diseño de interfaces verticales mediante el uso del contenedor `Column`, identificando la correcta aplicación de modificadores, recursos de imagen optimizados y la solución efectiva a problemas de desbordamiento mediante el uso de `LazyColumn`.

1.3.2. Objetivos Específicos

- Crear un proyecto funcional en Android Studio utilizando la plantilla *Empty Compose Activity*.
- Demostrar el comportamiento predeterminado de superposición de componentes de texto cuando se carece de un layout de organización.
- Implementar el componente `Column` y evaluar la concatenación de modificadores para tamaño completo, color de fondo, ancho máximo y alineación textual.
- Integrar imágenes optimizadas en formato WebP con descripciones accesibles para lectores de pantalla.
- Diagnosticar las limitaciones de la columna estándar ante el cambio de orientación de la pantalla y solucionar el desbordamiento mediante la transición a `LazyColumn`.

1.4. Justificación

Este trabajo me permite afianzar conceptos clave de la maquetación declarativa en el desarrollo Android moderno, comprendiendo la diferencia estructural y funcional entre contenedores estáticos y contenedores con capacidad de desplazamiento e interactividad dinámica.

2. Marco Teórico

2.1. Definición de Conceptos Clave

Column: Es el contenedor fundamental dentro de Jetpack Compose que organiza a sus elementos hijos de manera secuencial vertical, de arriba hacia abajo, equivalente al comportamiento clásico que utilizaba previamente en XML con un `LinearLayout` de orientación vertical.

Modifier: Constituye el objeto fundamental utilizado para personalizar el aspecto, tamaño, color, espaciado, comportamiento e interacciones de los componentes visuales en la interfaz declarativa.

Modificadores de Dimensión: Son indicadores específicos que me permiten definir si un contenedor o un elemento visual debe abarcar la totalidad de la superficie de la pantalla o únicamente expandirse a lo largo del ancho disponible del contenedor padre.

Formato de Imagen WebP: Representa un formato de compresión de imágenes altamente optimizado para el desarrollo móvil, el cual reduce significativamente el tamaño del archivo final sin comprometer la calidad visual ni el rendimiento en memoria.

LazyColumn: Es un componente de renderizado altamente eficiente que actúa como un contenedor con desplazamiento vertical. Conceptualmente equivale al patrón `RecyclerView` de la arquitectura clásica, con la ventaja de que solo compone y renderiza los elementos que son visibles actualmente en la pantalla del dispositivo.

2.2. Teorías y Antecedentes

Durante la preparación de esta práctica, analicé cómo el ciclo de vida y la renderización en Jetpack Compose optimizan el uso de recursos de hardware al emplear composables perezosos. Identifiqué que el sistema recalcula y redibuja la interfaz únicamente ante cambios reales en el estado de la aplicación, evitando reestructuraciones innecesarias en la jerarquía de vistas.

3. Desarrollo

3.1. Análisis del Cambio de Paradigma: Column vs. LinearLayout

A continuación, presento el análisis comparativo que elaboré para contrastar el enfoque tradicional basado en archivos de diseño XML con el esquema declarativo moderno que utilicé en esta práctica:

Tabla 1: Comparativa entre Column en Compose y LinearLayout vertical

Criterio de Comparación	**Layout Clásico (XML)**	**Jetpack Compose (Declarativo)**
Componente Equivalente	LinearLayout con orientación vertical	Contenedor dinámico `Column`
Definición de Estilos	Atributos XML individuales o archivos de estilo	Concatenación de funciones sobre el objeto `Modifier`
Renderizado de Listas	Requiere acoplar un RecyclerView y un Adapter	Sustitución directa por el componente `LazyColumn`
Invocación en Código	Inflado de Vistas / Enlazado de datos (View Binding)	Invocación directa de funciones composables

3.2. Guía Paso a Paso y Desarrollo de la Práctica

Paso 1: Creación del Proyecto Base

Configuré un nuevo proyecto de desarrollo denominado "column application" mediante la plantilla nativa Empty Compose Activity dentro del entorno Android Studio.



Figura 1: Creación del proyecto "column application" en Android Studio

Paso 2: Limpieza del Entorno y Definición de la Función Principal

Removí todo el código generado por defecto dentro del bloque de contenido principal de la actividad y creé mi propia función composable contenedora junto con su respectiva función de vista previa.

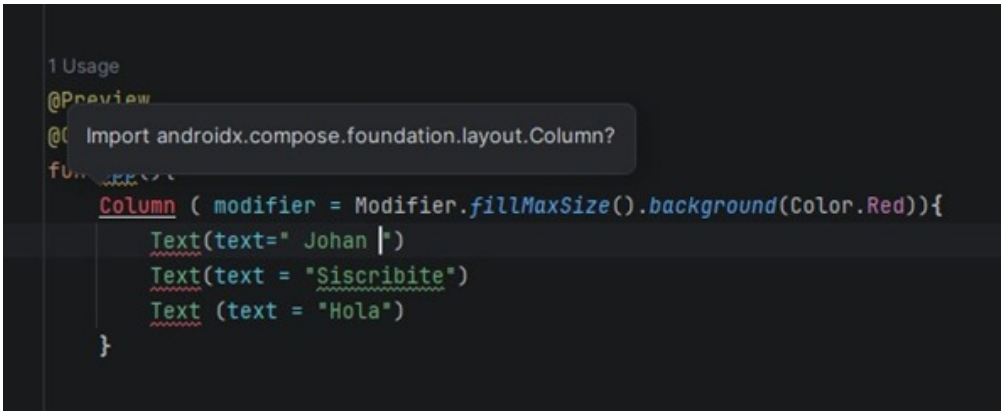


Figura 2: Configuración inicial de la función composable y vista previa

Paso 3: Evidencia de Superposición de Elementos

Coloqué tres componentes de texto independientes sin envolverlos en ningún contenedor de maquetación, comprobando experimentalmente cómo se encimaban en la esquina superior izquierda de la pantalla.



Figura 3: Comprobación del encimado de textos al no usar un contenedor de maquetación

Paso 4: Implementación del Contenedor Column

Encapsulé los tres componentes de texto dentro de una columna, logrando que los elementos se organizaran automáticamente de forma secuencial uno debajo del otro.



Figura 4: Organización vertical exitosa de los elementos mediante Column

Paso 5: Expansión del Contenedor a Pantalla Completa

Modifiqué la estructura de la columna agregando el modificador necesario para forzar al contenedor a ocupar el tamaño máximo disponible en la pantalla.



Figura 5: Aplicación del modificador para expandir la columna a pantalla completa

Paso 6: Personalización del Fondo Visual

Apliqué un color de fondo rojo a toda la superficie de la columna concatenando un modificador de color dentro de la cadena de configuración visual.

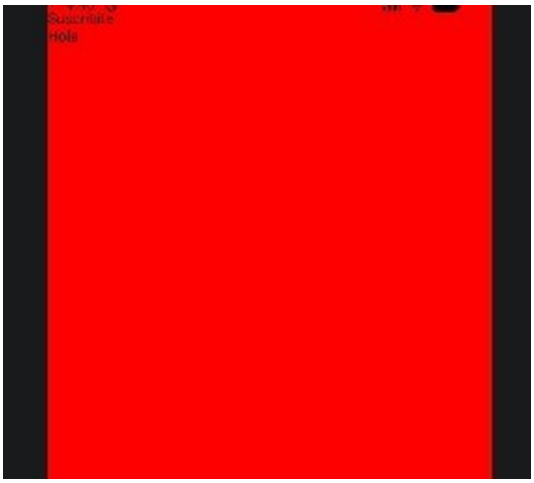
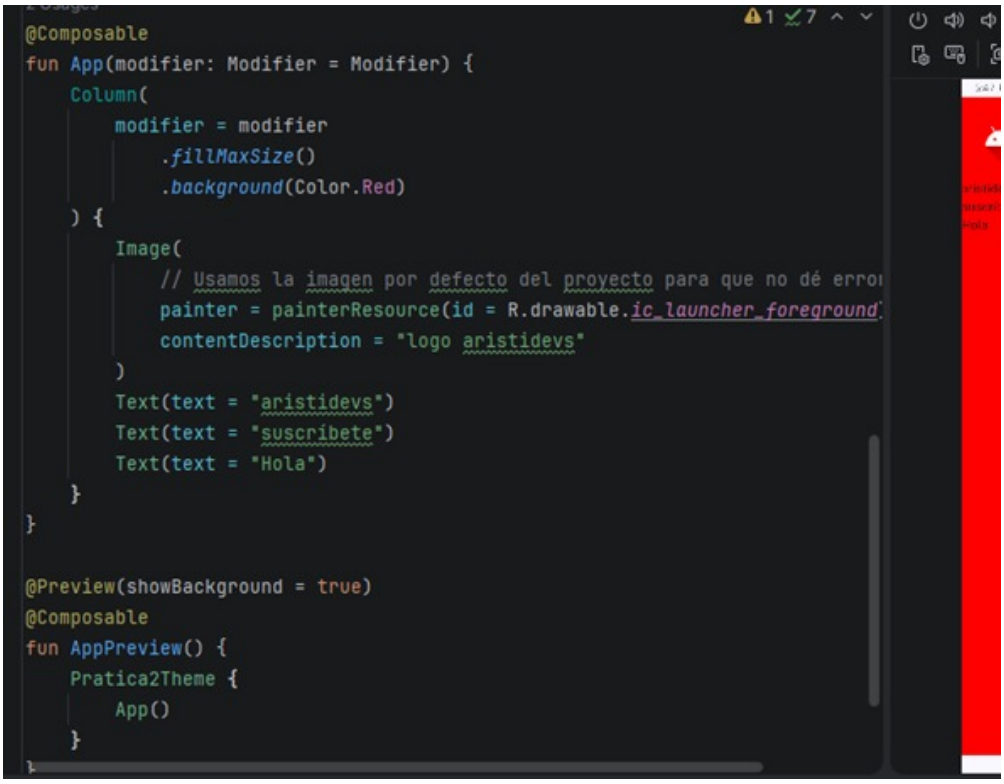


Figura 6: Asignación de color de fondo al contenedor mediante modificadores

Paso 7: Importación de Recurso Gráfico WebP

Añadí una imagen previamente convertida al formato optimizado WebP dentro del directorio de recursos gráficos de mi proyecto.



```
@Composable
fun App(modifier: Modifier = Modifier) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .background(Color.Red)
    ) {
        Image(
            // Usamos la imagen por defecto del proyecto para que no dé erro
            painter = painterResource(id = R.drawable.ic_launcher_foreground
            contentDescription = "logo aristidevs"
        )
        Text(text = "aristidevs")
        Text(text = "suscribete")
        Text(text = "Hola")
    }
}

@Preview(showBackground = true)
@Composable
fun AppPreview() {
    Pratica2Theme {
        App()
    }
}
```

Figura 7: Incorporación del archivo de imagen WebP en la carpeta drawable

Paso 8: Inclusión de la Imagen con Descripción Accesible

Inserté el componente gráfico de imagen dentro de la jerarquía de la columna, especificando la fuente del recurso visual y asignando una descripción textual explícita para lectores de pantalla.

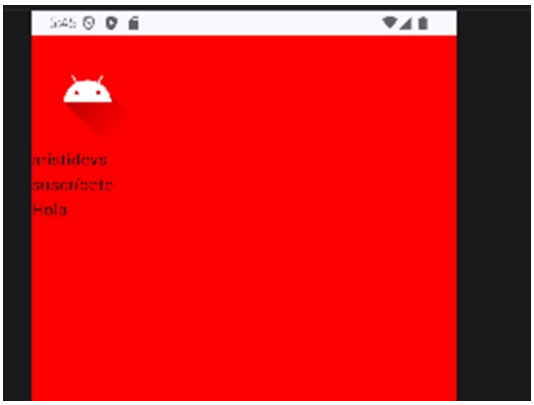


Figura 8: Visualización de la imagen integrada en la jerarquía vertical

Paso 9: Estilizado de Textos y Colores de Fuente

Modifiqué el tamaño de la tipografía del título asignándole una dimensión de 32 puntos de escala y cambié el color del texto a blanco para garantizar un alto contraste con el fondo.

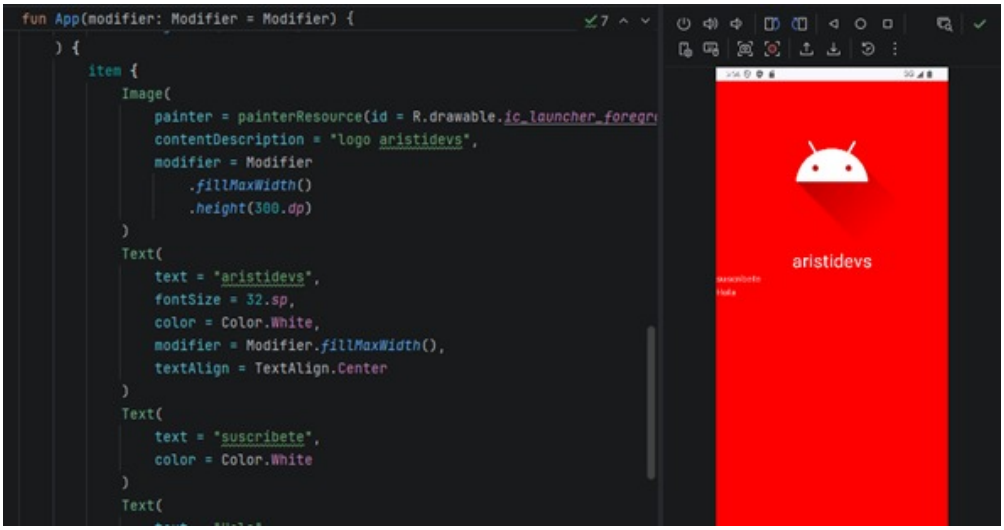


Figura 9: Personalización de tamaño y color en los textos de la interfaz

Paso 10: Expansión Horizontal y Alineación de Texto

Apliqué el modificador de expansión de ancho máximo al elemento de título y definí la alineación interna del texto para ubicarlo exactamente en el centro horizontal.

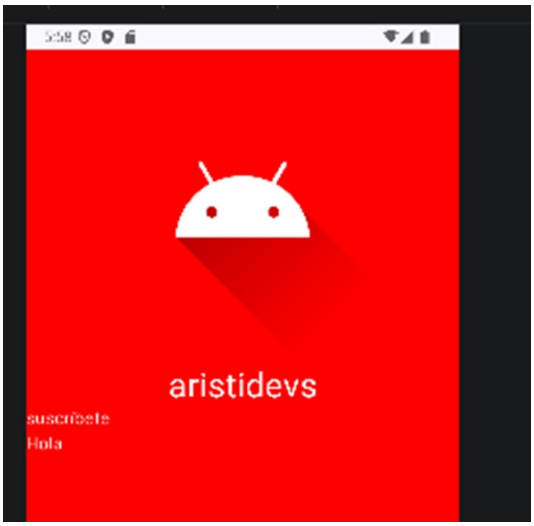


Figura 10: Centrado del título mediante expansión de ancho y alineación de texto

3.3. Explicación Narrativa del Comportamiento Visual y Estructura

Durante la realización de la práctica, analicé detalladamente cómo la secuencia de invocación en los modificadores altera la interpretación del renderizado. Al concatenar ajustes de tamaño antes que los de color, aseguré que la propiedad cromática cubriera toda la superficie extendida y no únicamente la región ocupada por el texto inicial.

Comprobé también que la columna tradicional es ideal para vistas con una cantidad pequeña y fija de elementos visuales. Sin embargo, cuando el espacio vertical disponible se reduce —como sucede al cambiar la orientación del dispositivo a modo horizontal—, la columna estándar carece de la capacidad implícita de desplazamiento, lo que genera un recorte indeseado de los componentes inferiores.

Al migrar hacia la estructura de `LazyColumn`, solucioné de manera inmediata esta limitación. La columna perezosa crea únicamente las vistas visibles en el área activa de la pantalla, optimizando drásticamente el consumo de memoria RAM y permitiendo una interacción de desplazamiento suave y continua en cualquier orientación del dispositivo. Finalmente, la integración del formato WebP me permitió constatar una carga de archivos más ágil, manteniendo la nitidez gráfica sin penalizar el rendimiento global de la aplicación.

4. Conclusiones

- Concluyo que el paradigma declarativo de Jetpack Compose simplifica significativamente la construcción de interfaces de usuario verticales al eliminar la necesidad de definir archivos XML independientes.
- Identifiqué que el orden en que concateno los modificadores sobre un componente visual afecta de manera directa su posicionamiento y apariencia en la pantalla.
- Comprobé la importancia crítica de anticipar cambios en la orientación del dispositivo, reconociendo que `LazyColumn` es la solución adecuada para evitar problemas de desbordamiento de contenido.
- Determiné que el uso de formatos modernos de imagen como WebP contribuye directamente a la optimización de recursos y al alto desempeño de las aplicaciones móviles.

5. Anexos

Anexo A: Referencia del Material de Apoyo

Para el desarrollo y validación paso a paso de esta práctica, me fundamenté en los conceptos presentados en el tutorial especializado de desarrollo Android:

- **Referencia del recurso:** Android Developers Video Tutorials — Layouts and Modifiers in Jetpack Compose.
- **Enlace de acceso:** https://developer.android.com/jetpack/compose/layouts

Enlace: https://github.com/Elprojobra/Aplicaciones-moviles/upload

6. Referencias

Android Developers. (2024). *Jetpack Compose Layouts*. Google. https://developer.android.com/jetpack/compose/layouts

Android Developers. (2024). *Lists and Performance in Compose*. Google. https://developer.android.com/jetpack/compose/lists

Google Developers. (2023). *WebP Image Format Documentation*. https://developers.google.com/speed/webp